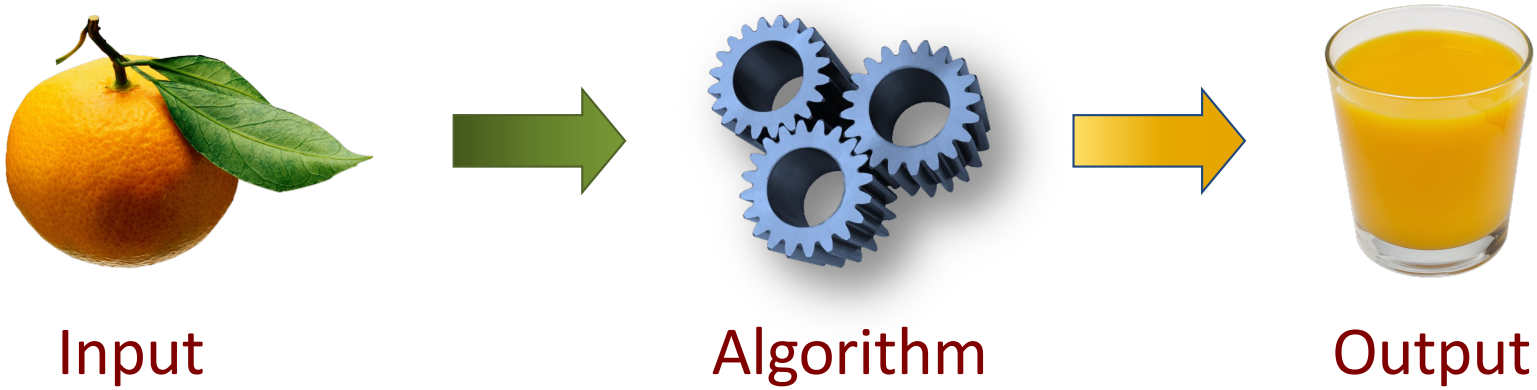
Analysis Tools

- Counting Primitive Operations
- Asymptotic Analysis
  - and examples



What are We Interested In?

Data structure – a systematic way of organising and accessing data

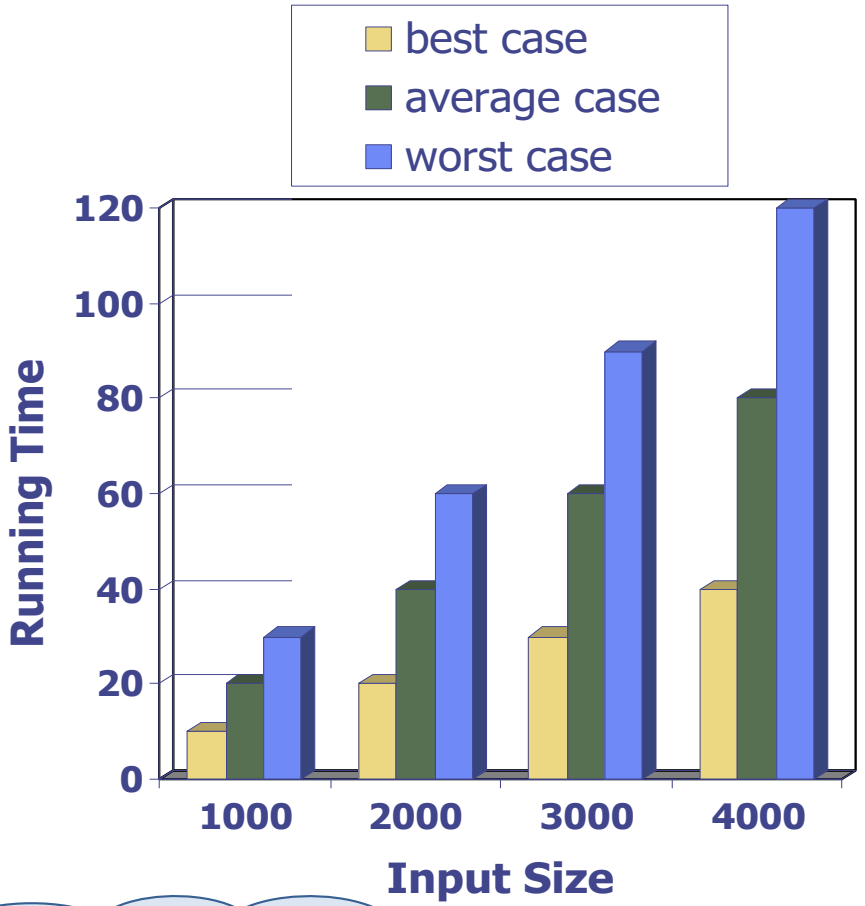
Algorithm – a step-by-step procedure for performing some task in a finite amount of time

- Running time
- Space usage
- How does time and space increase with size of INPUT?

Why does more memory speed up your computer?

Running Time

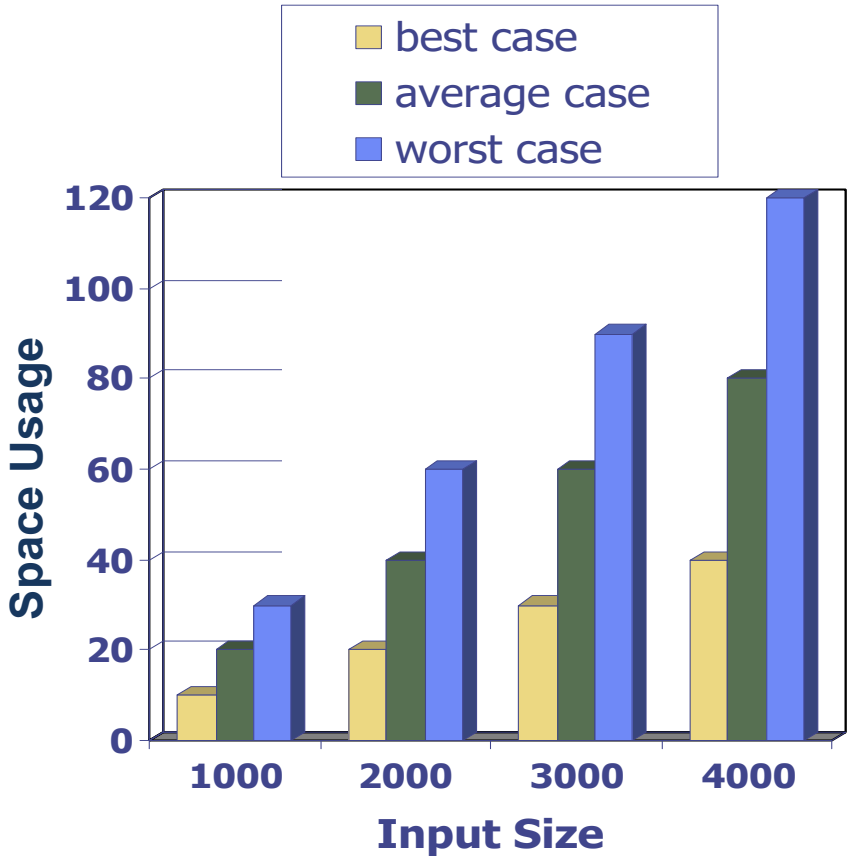
- Most algorithms transform input data into output data
- Running time of an algorithm typically grows with input size
- *Average* case time is often difficult to determine
- We focus on the *worst* case running time
  - easier to analyse
  - crucial for games, finance, robotics, eScience, …



What does "average case" really mean? And why "difficult"?

Space Usage

- Algorithms require space to store intermediate results
  - Space usage can vary dramatically depending on
    - algorithm implementation
    - iteration or recursion
  - Average-case space usage is often difficult to determine
  - We also focus on the worst-case space usage
    - easier to analyse
    - *crucial* for embedded systems
- Data structures need space to hold data elements



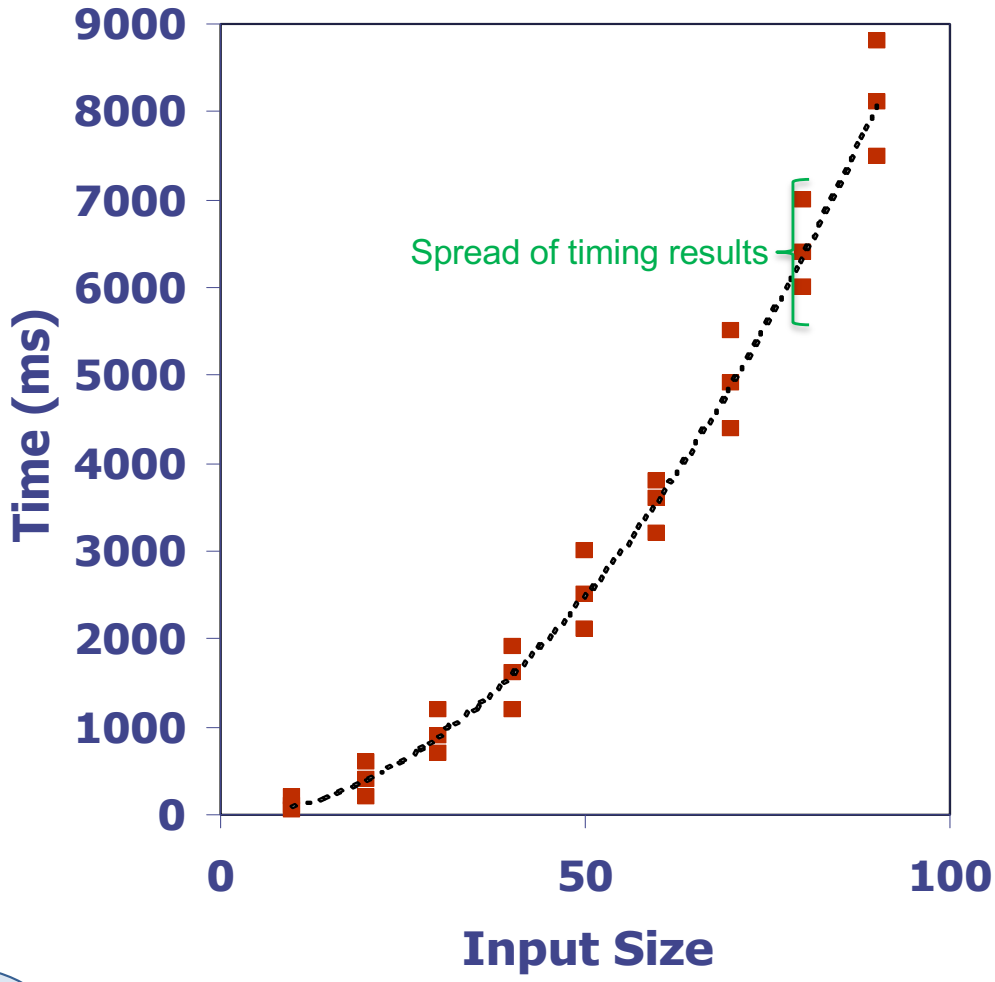
A probability distribution may be informative

How Can We Analyse Algorithms?

- Experimental (empirical) studies
- Theoretical analysis

Experimental Studies

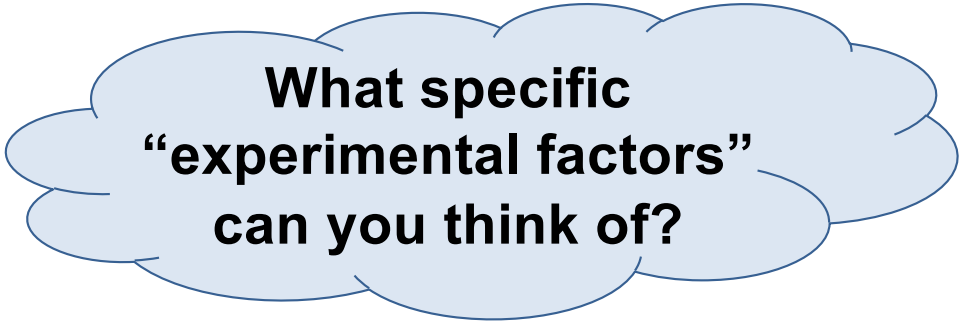
- Write program implementing the algorithm
- Run program with inputs of varying size and composition
- Use a method like *System.currentTimeMillis()* to measure actual running time
- Plot the results



Is there a function describing the time usage of this program?

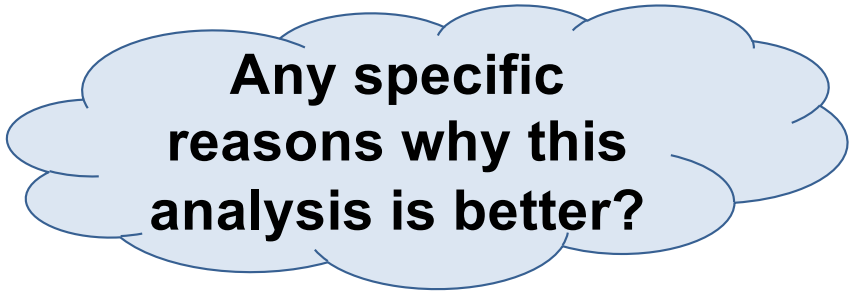
Limitations of Experiments

- Need to implement algorithm
  - may be difficult (and the algorithm may prove inadequate)
- Results may not be indicative of the running time on other inputs not included in the experiment
- Comparing algorithms requires the same hardware and software environments



Theoretical Analysis

- Use a high-level description of the algorithm
  - instead of an implementation
- Characterises running time as a function of input size, *n*
- Takes into account all possible inputs
  - at least those that are “bad”
- Evaluation is independent of the hardware / software environment



How Do We Perform Theoretical Analysis?

1. Express algorithm as pseudo-code
2. Count primitive operations
3. Describe algorithm as f(*n*)
   - function of *n*
     - property of input with significant impact on performance
4. Perform asymptotic analysis
   - express in asymptotic notation

Pseudo-Code

- High-level description of an algorithm
- More structured than English prose
- Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design details

Pseudo-Code Details

- Control flow
  - **if** … **then** … [**else** …]
  - **while** … **do** …
  - **repeat** … **until** …
  - **for** … **do** …
  - Indentation replaces braces
- Method declaration

  Algorithm ***method*** (***arg*** [, ***arg***…])
  Input …
  Output …

- Method call

  method (***arg*** [, ***arg***…])
- Return value

  return *expression*
- Expressions

  ← *(Assignment)*
  - like = in Java

  = *(Equality testing)*
  - like == in Java
- Mathematical formatting

  n^2

Find Max Element of an Array

```
Algorithm arrayMax(A, n)
    Input array A of n integers
    Output maximum element of A

    currentMax ← A[0]
    for i ← 1 to n – 1 do
        if A[i] > currentMax then
            currentMax ← A[i]
        { increment counter i }
    return currentMax
```

Pseudo-code

```
public int arrayMax(int[] A) {
    int currentMax = A[0];

    for(int i=1; i < A.length; i++) {
        if (A[i] > currentMax) {
            currentMax = A[i];
        }  // increment counter i
    }
    return currentMax;
}
```

Java code

Primitive Operations

- Basic computations performed by an algorithm
- Identifiable in pseudo-code
- Largely independent from the programming language
- Exact definition not important
  - later we will see why
- Assumed to take a constant amount of time

- Examples
  - Assigning a value to a variable
  - Calling a method
  - Performing an arithmetic operation
  - Evaluating a conditional
  - Indexing into an array
  - Following an object reference
  - Returning from a method

Counting Primitive Operations

Assignment

int *num* = 10; 1 operation

1. Assigning a value to a variable

int *num* = *A*[10]; 2 operations

1. Indexing into an array
2. Assigning a value to a variable

Counting Primitive Operations

Control Flow

```
if (num < 10)                    1 operation
{
  ...  add operations
}
```

```
if (num < A[10])                 2 operations
{
  ...  add operations
}
```

Counting Primitive Operations

Loops

```
while (i < 10)                    1 operation per iteration + 1
{
  ... counted operations times loop
}


do
{
  ... counted operations times loop
} while (i < 10)                  1 operation per iteration
```

Counting Primitive Operations

Loops

```
for (int i=0; i < n; i++)                    1 + (n + 1)
{
   ... let counted operations = X            n · X
}  // count increment                        n
```

- "int *i*=0" is executed once
- "*i* < *n*" conditional is evaluated *n* + 1 times
- "*i*++" increment is evaluated *n* times
- *X* instructions within loop are evaluated *n* times

Describe as f(n) – Function of n

- By inspecting the pseudo-code, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm *arrayMax*(*A*, *n*)	# operations
currentMax ← *A*[0]	2
for *i* ← 1 to *n* - 1 do	1 + 2 · *n* or 2 + *n*
if *A*[*i*] > *currentMax* then	2 · (*n* - 1)
currentMax ← *A*[*i*]	2 · (*n* - 1)
{ increment counter *i* }	*n* - 1
return *currentMax*	1
Total:	7 · *n* + *lower order terms*

Can we safely ignore lower-order terms like this?

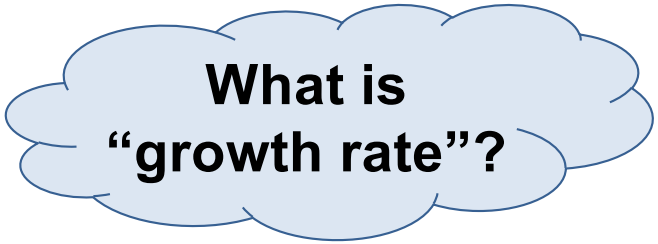
Estimating Running Time

- ***arrayMax*** executes $7n$ primitive operations in the worst case
  - a = time taken by the fastest primitive operation
  - b = time taken by the slowest primitive operation
- Let $T(n)$ be worst-case time of ***arrayMax***
  - $a \cdot 7n \leq T(n) \leq b \cdot 7n$

- Run time $T(n)$, is bounded by two linear functions

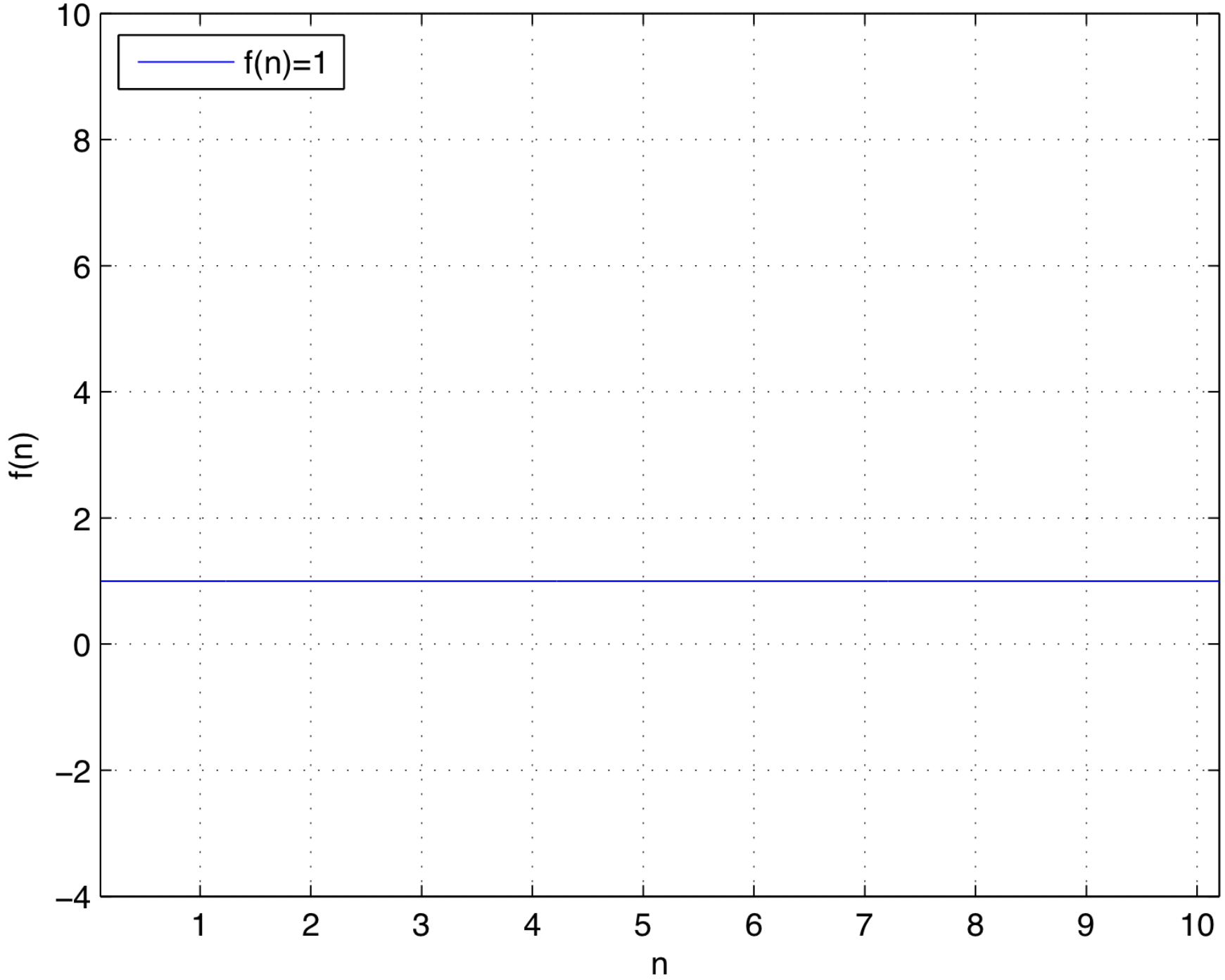
Growth Rate of Running Time

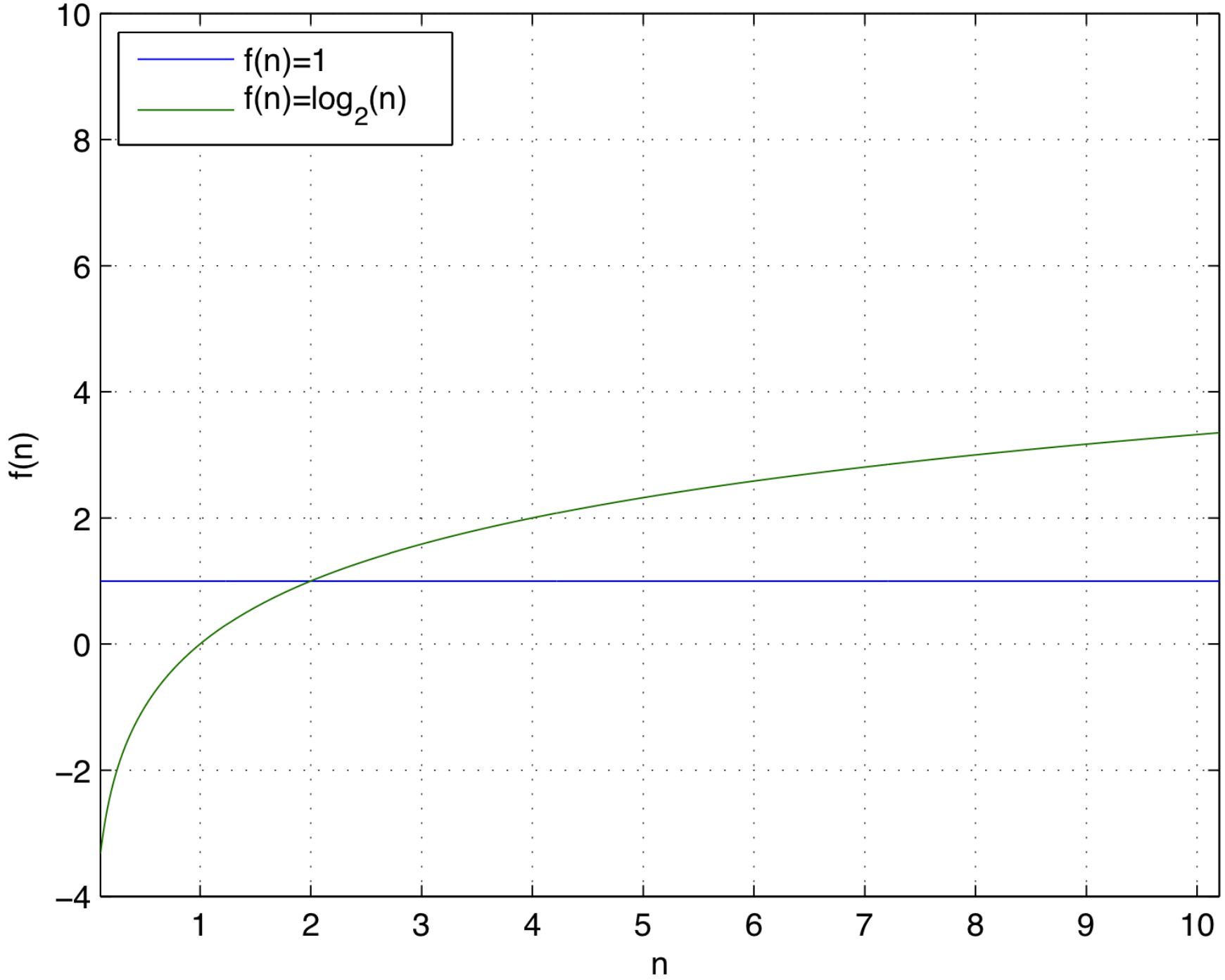
- Changing hardware/software environment
  - affects $T(n)$ by a constant factor
  - does not alter the *growth rate* of $T(n)$
- Growth rate of the running time $T(n)$ is an intrinsic property of algorithm ***arrayMax***

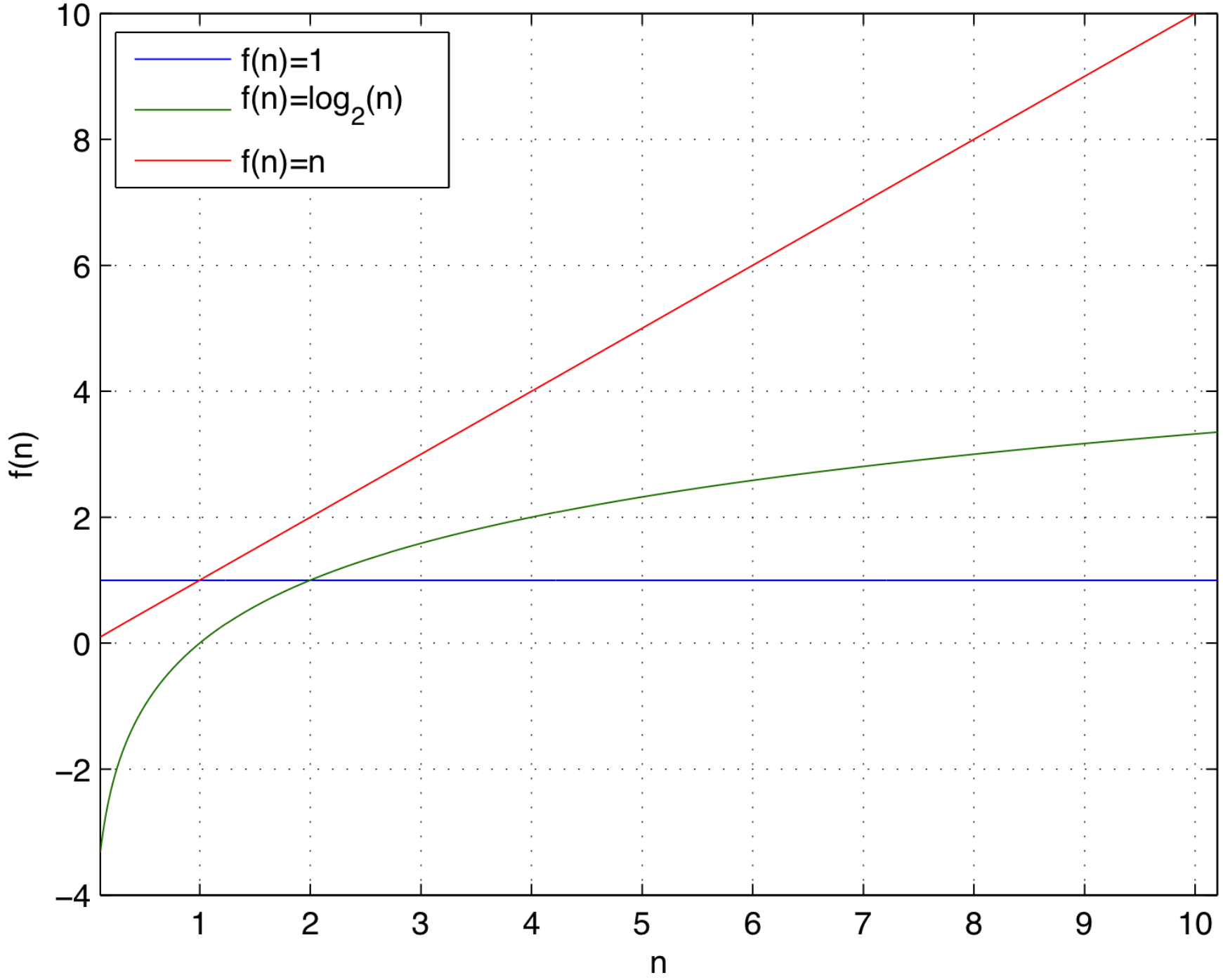


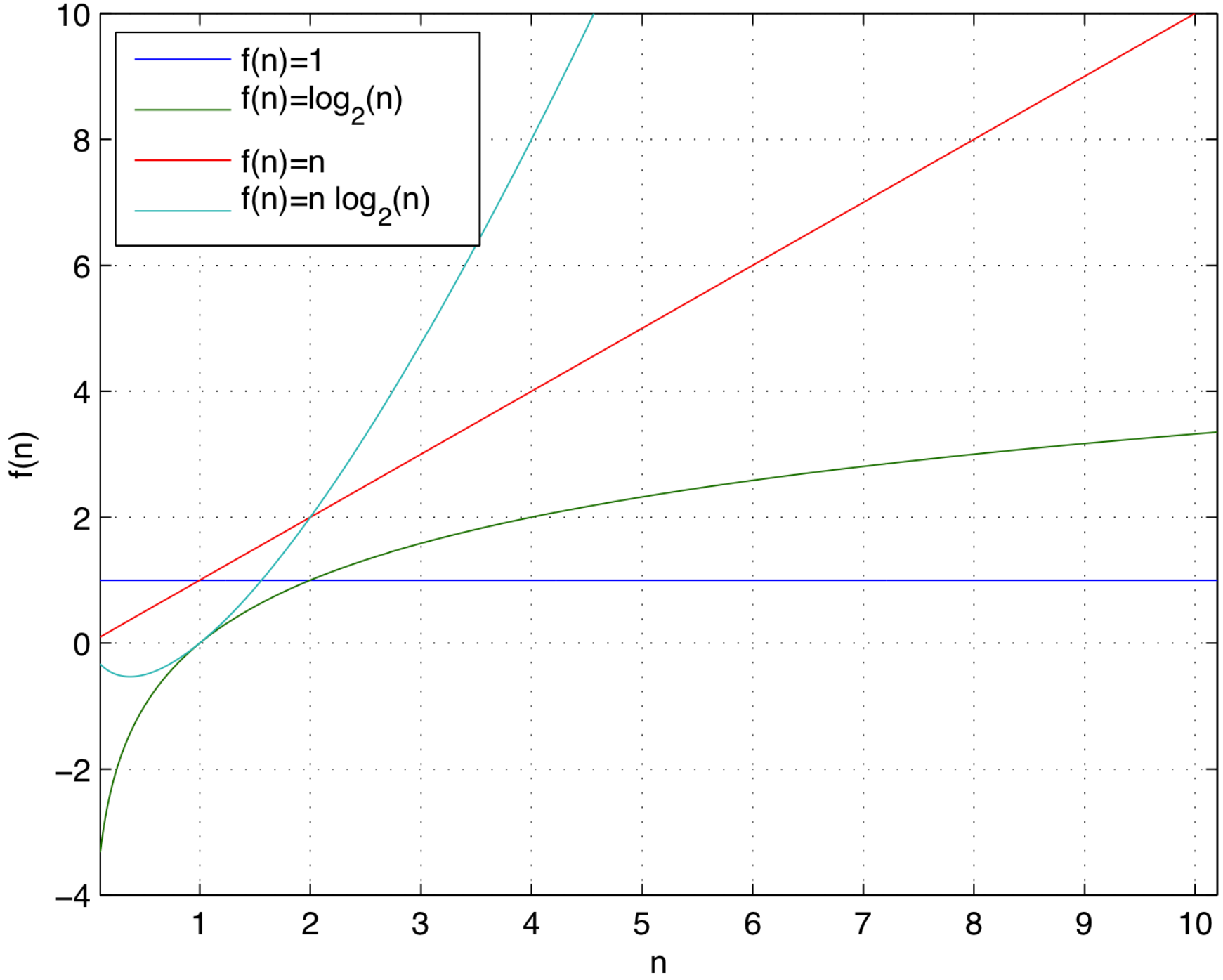
Seven Important Functions

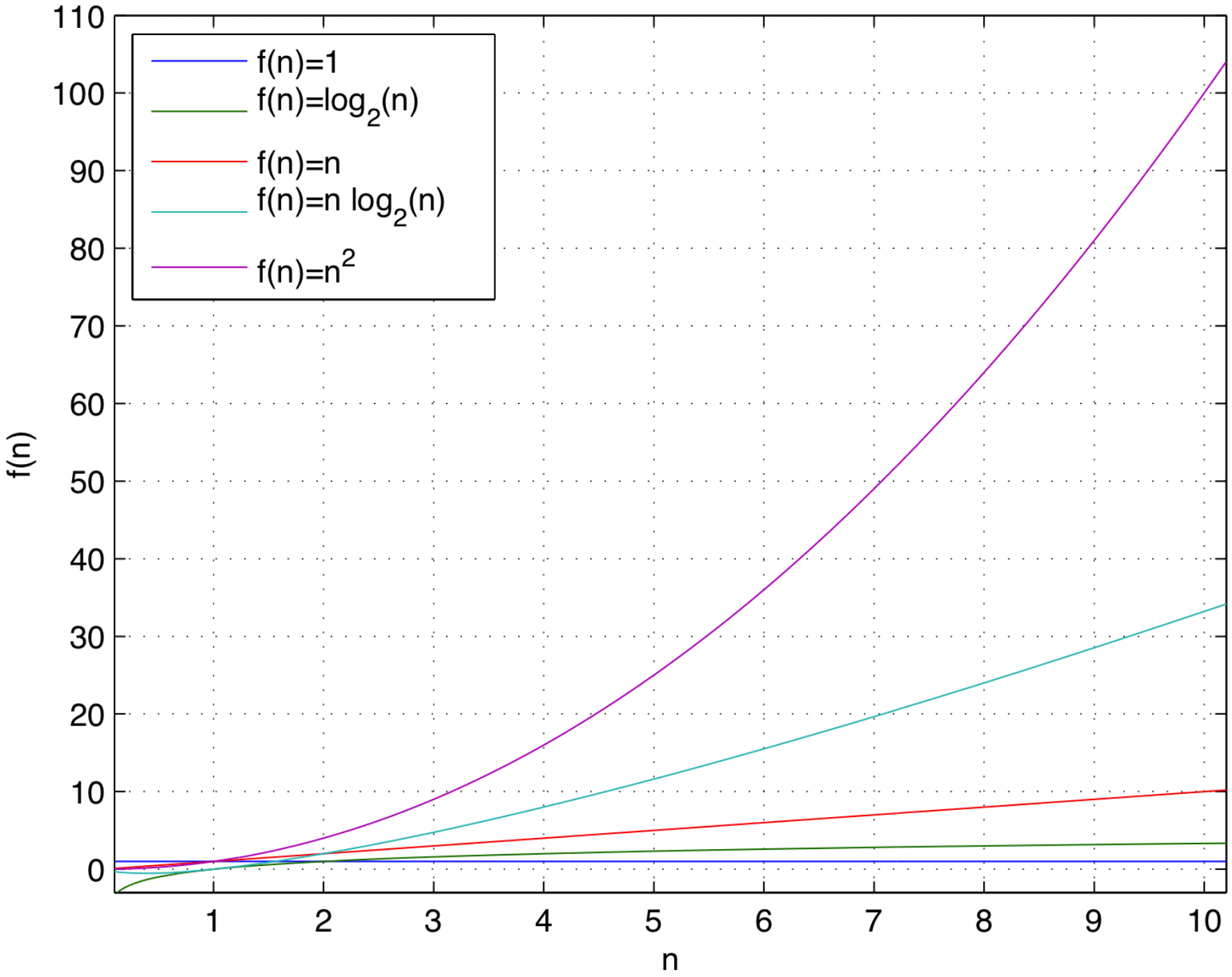
- Seven functions that often appear in algorithm analysis
  - Constant $\approx$ 1
  - Logarithmic $\approx$ $\log_2 n$
  - Linear $\approx$ n
  - N-Log-N $\approx$ $n \log_2 n$
  - Quadratic $\approx$ n^2
  - Cubic $\approx$ n^3
  - Exponential $\approx$ 2^n
  - Factorial $\approx$ $n!$ // not so often

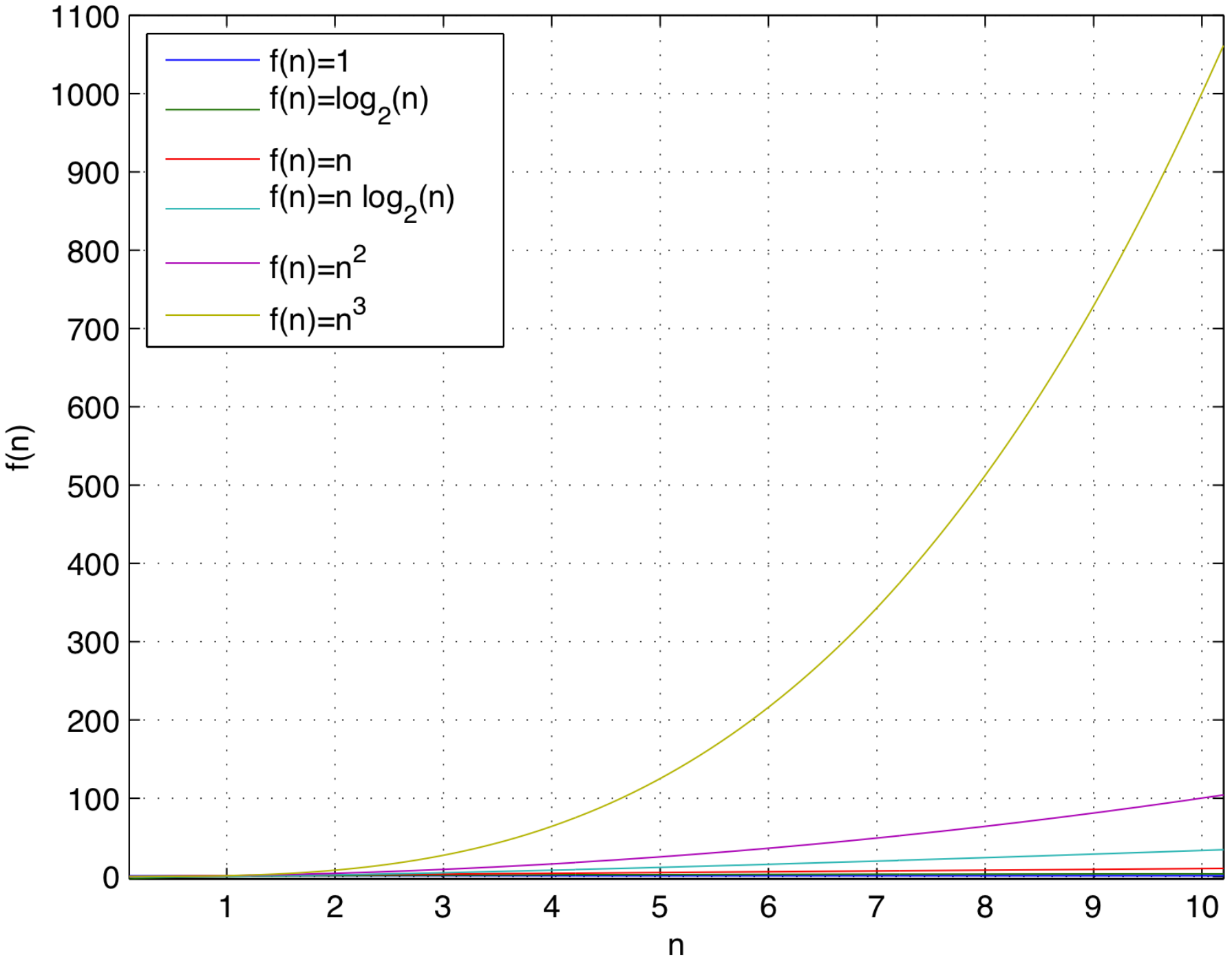

f(n)=1
10
8
6
4
2
0
−2
−4
f(n)
1
2
3
4
5
6
7
8
9
10
n

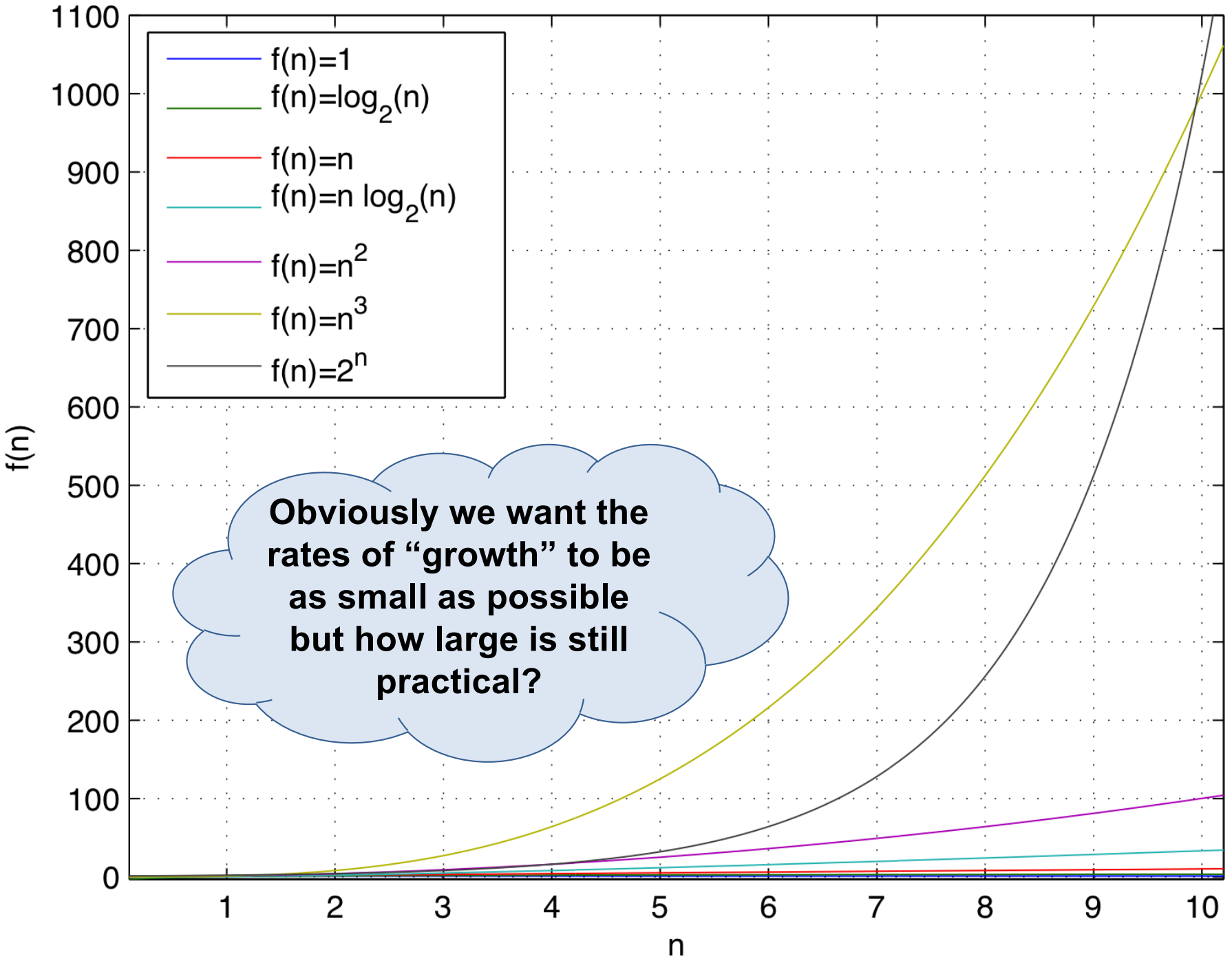

f(n)=1
f(n)=log2(n)
10
8
6
4
2
0
−2
−4
f(n)
1
2
3
4
5
6
7
8
9
10
n


f(n)=1
f(n)=log2(n)
f(n)=n
10
8
6
4
2
0
−2
−4
f(n)
1
2
3
4
5
6
7
8
9
10
n


f(n)=1
f(n)=log2(n)
f(n)=n
f(n)=n log2(n)
f(n)
n
10
8
6
4
2
0
−2
−4
1
2
3
4
5
6
7
8
9
10


f(n)=1
f(n)=log2(n)
f(n)=n
f(n)=n log2(n)
f(n)=n2
f(n)
n
0
10
20
30
40
50
60
70
80
90
100
110
1
2
3
4
5
6
7
8
9
10


f(n)=1
f(n)=log2(n)
f(n)=n
f(n)=n log2(n)
f(n)=n2
f(n)=n3
f(n)
n
0
100
200
300
400
500
600
700
800
900
1000
1100
1
2
3
4
5
6
7
8
9
10

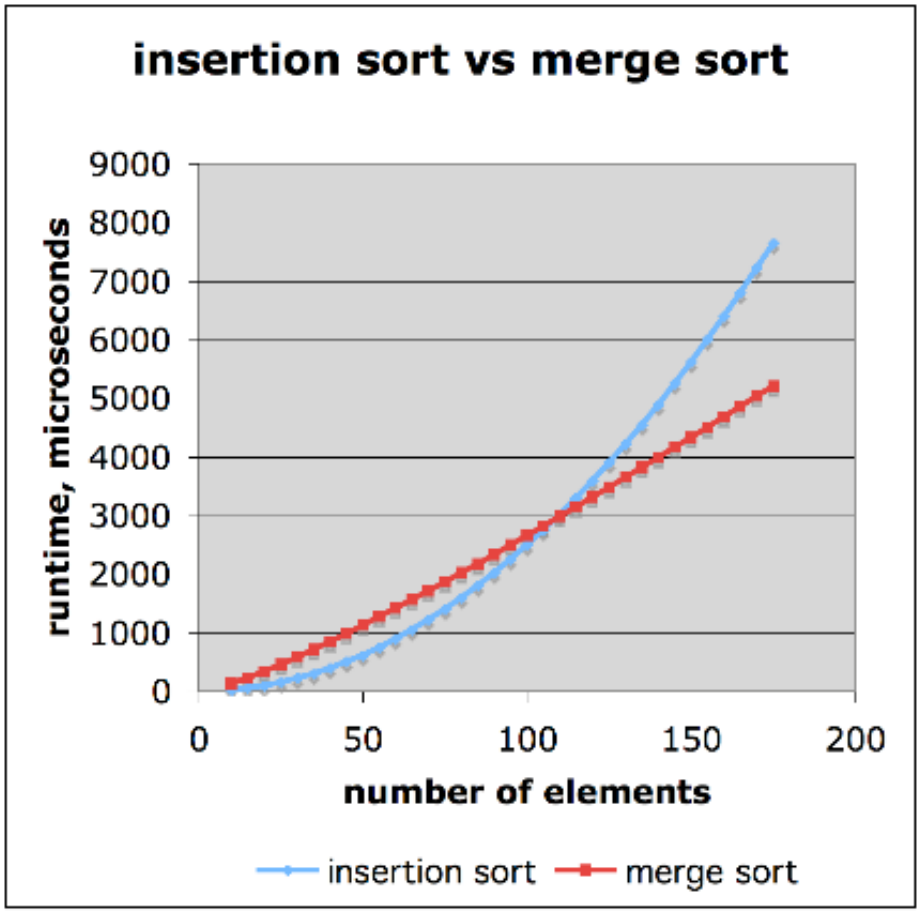

f(n)=1
f(n)=log2(n)
f(n)=n
f(n)=n log2(n)
f(n)=n2
f(n)=n3
f(n)=2n
Obviously we want the rates of "growth" to be as small as possible but how large is still practical?
f(n)
n
0
100
200
300
400
500
600
700
800
900
1000
1100
1
2
3
4
5
6
7
8
9
10

If we count only the number of times “pick” is executed and that takes 10^-9s (1/1billion ~ GFlop), it would take 585 years to run this harmless looking loop.

```
n ← 64
k ← 1

for i ← 0 to n-1 do
   for j ← 0 to k-1 do
      pick()
      { increment counter j }
   k ← k * 2
   { increment counter i }
```

Comparison of Two Algorithms



Insertion sort is

$n^2 / 4$

Merge sort is

$2\ n \log_2(n)$

Sort a million items?

insertion sort takes roughly 70 hours

while

merge sort takes roughly 40 seconds

This on a slow machine, but if 100 x as fast then it's 40 minutes versus less than 0.5 seconds

The plot is deceptive. Can you see why?

We illustrate the growth rates of some important functions in Table 4.2.

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,096	262,144	1.84×10^{19}
128	7	128	896	16,384	2,097,152	3.40×10^{38}
256	8	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	512	4,608	262,144	134,217,728	1.34×10^{154}

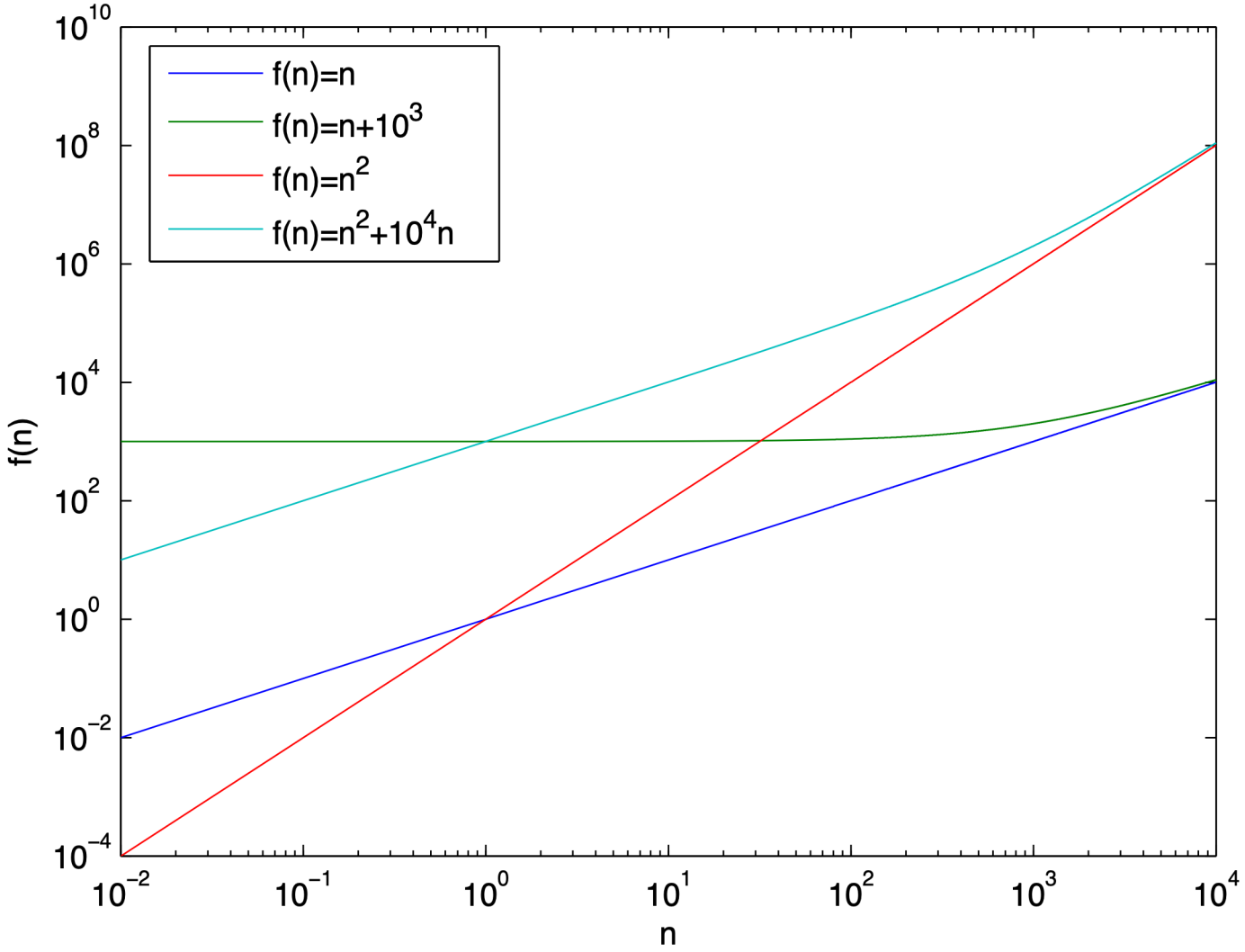
Table 4.2: Selected values of fundamental functions in algorithm analysis.

We further illustrate the importance of the asymptotic viewpoint in Table 4.3. This table explores the maximum size allowed for an input instance that is processed by an algorithm in 1 second, 1 minute, and 1 hour. It shows the importance of good algorithm design, because an asymptotically slow algorithm is beaten in the long run by an asymptotically faster algorithm, even if the constant factor for the asymptotically faster algorithm is worse.

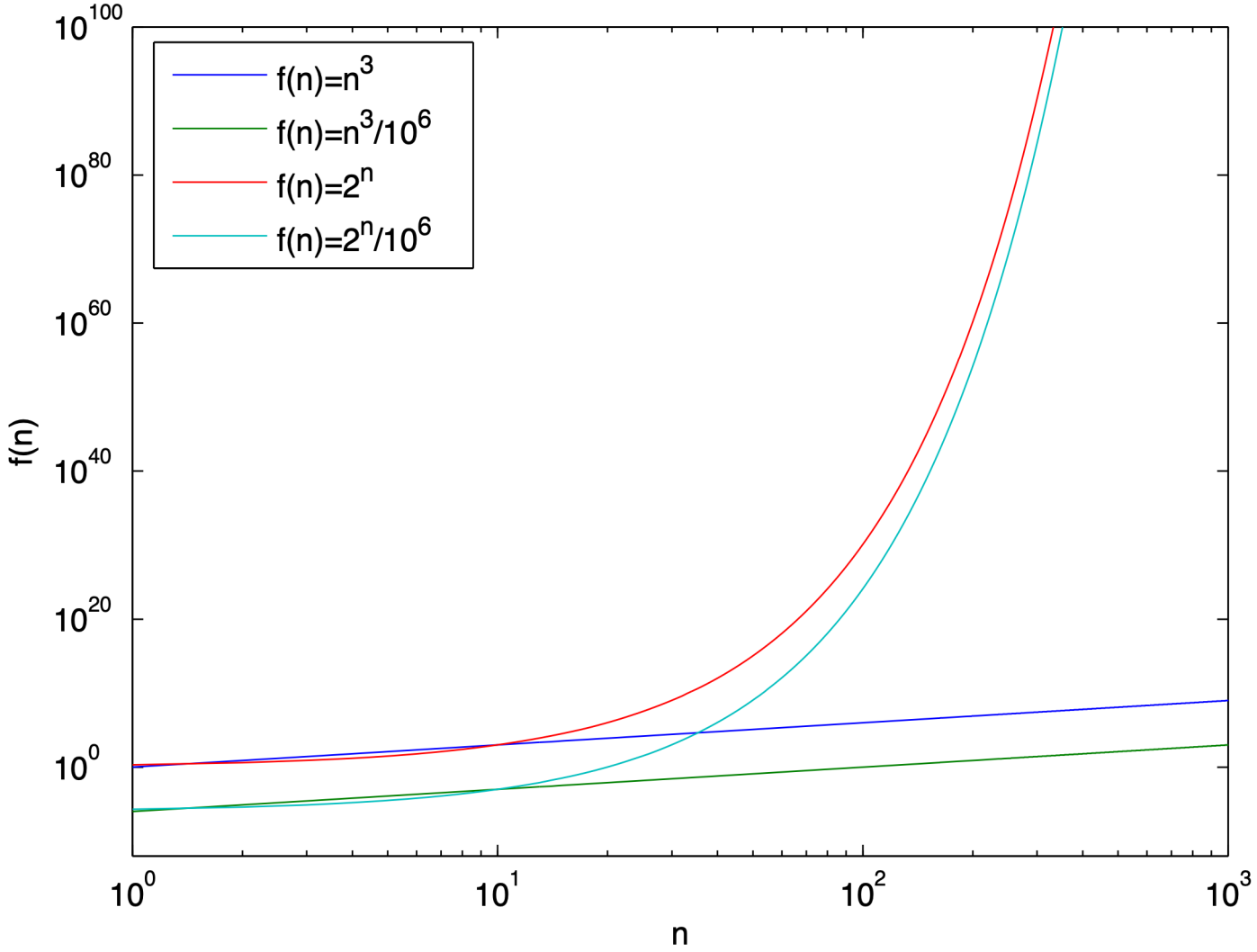
Running Time (μs)	Maximum Problem Size (n)		
	1 second	1 minute	1 hour
$400n$	2,500	150,000	9,000,000
$2n^2$	707	5,477	42,426
2^n	19	25	31

Table 4.3: Maximum size of a problem that can be solved in 1 second, 1 minute, and 1 hour, for various running times measured in microseconds.

What Difference do *Lower-Order Terms* Make?



What Difference Does a *Constant Factor* Make?



Event	Seconds	Scientific Notation
brain cell firing	0.01	1.00E-02
1 heart beat	1	1.00E+00
1 hour	3,600	3.60E+03
1 day	86,400	8.64E+04
1 week	604,800	6.05E+05
1 month	2,592,000	2.59E+06
1 year	31,557,600	3.16E+07
Human life span (~100 years)	3,155,760,000	3.16E+09
Millennium (1000 years)	31,557,600,000	3.16E+10
Time since the last ice age (10,000 years)	315,576,000,000	3.16E+11
Time since dinosaurs (60M years)	1,893,456,000,000,000	1.89E+15
Age of universe = 14 billion years	441,806,400,000,000,000	4.42E+17

Operation	Time
L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1KB w/ cheap algorithm	3,000 ns
Send 2KB over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek 10ms; was 30ms in 1980!!	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA 0.25s	150,000,000 ns

Asymptotic Analysis

- Big-O notation
- Relatives of Big-O
- Examples

Asymptotic Analysis

"In pure and applied mathematics, particularly the analysis of algorithms, real analysis, and engineering, asymptotic analysis is a method of describing limiting behaviour."

http://en.wikipedia.org/wiki/Asymptotic_analysis

"In computational complexity theory, asymptotic computational complexity is the usage of *asymptotic analysis* for the estimation of computational complexity of algorithms and computational problems, commonly associated with the usage of the big O notation."

http://en.wikipedia.org/wiki/Asymptotic_computational_complexity

Analysis Process

- To perform an asymptotic analysis of the worst-case running time of an algorithm
  - find the worst-case number of primitive operations executed as a function of the input size – f(*n*)
    - since constant factors and lower-order terms do not affect the growth rate for large *n* they are usually disregarded when counting primitive operations
  - express this function with big-O notation

Big-O Notation

- Given functions $f(n)$ and $g(n)$, we say that

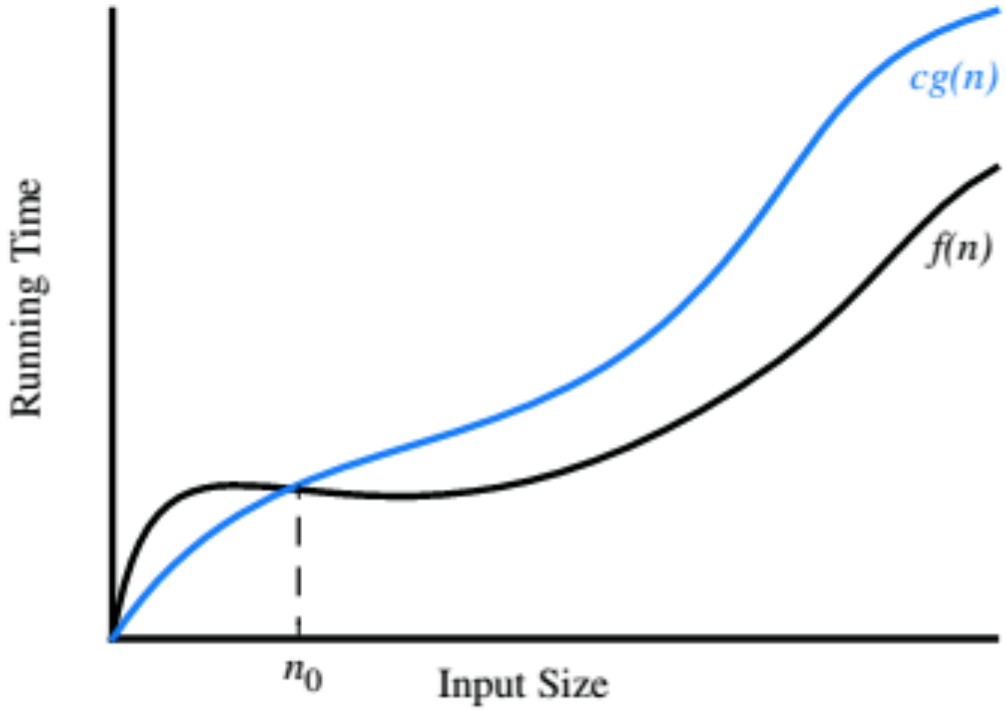
  $f(n)$ is $O(g(n))$

  if there are positive constants c and n_0 such that

  $f(n) \leq c \cdot g(n)$ for $n \geq n_0$ // i.e. large enough n
- Another way of saying the same thing is

  $f(n)$ is a member of the set of functions $O(g(n))$

Illustrating Big-O Notation

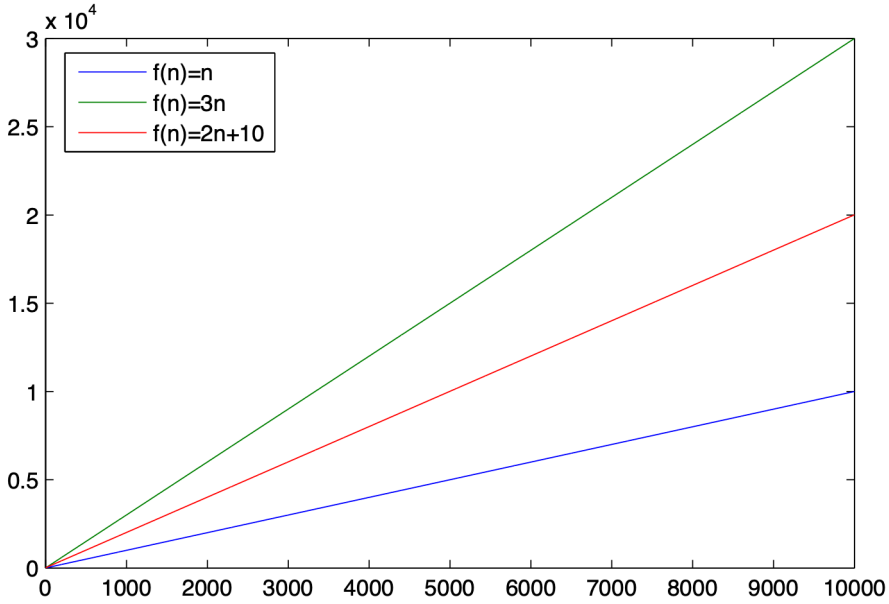


- $f(n)$ is $O(g(n))$, since
- $f(n) \leq c \cdot g(n)$ when $n \geq n_0$

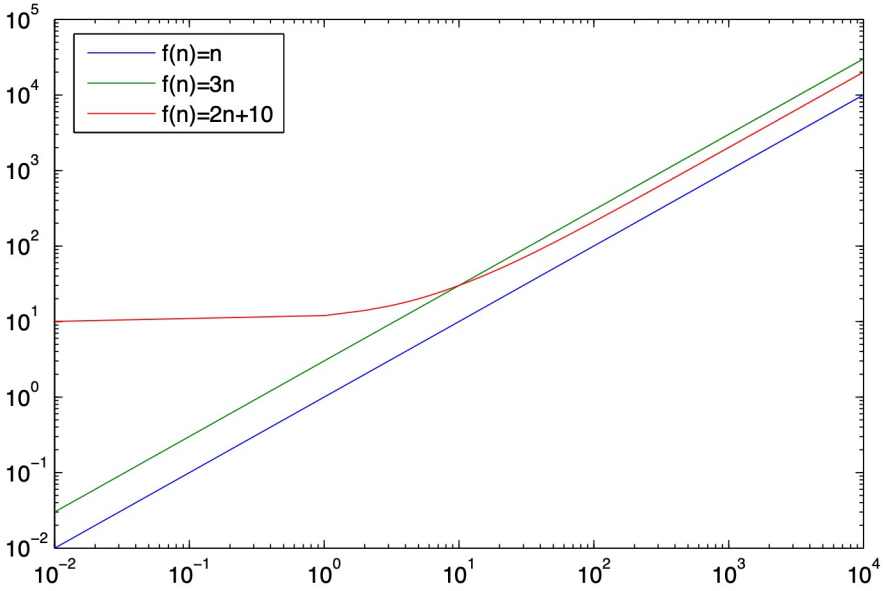
From Goodrich and Tamassia, *Data Structures & Algorithms.*

Big-O Example

- Given functions $f(n)$ and $g(n)$, we say that
  - $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
- $f(n) = 2n + 10$
- $g(n) = n$



- Proof: $2n + 10$ is $O(n)$
  - $2n + 10 \leq c \cdot n$
  - $10 \leq c \cdot n - 2n$
  - $c \cdot n - 2n \geq 10$
  - $(c - 2)\, n \geq 10$
  - $n \geq 10 \div (c - 2)$
  - Pick $c = 3$ and $n_0 = 10$



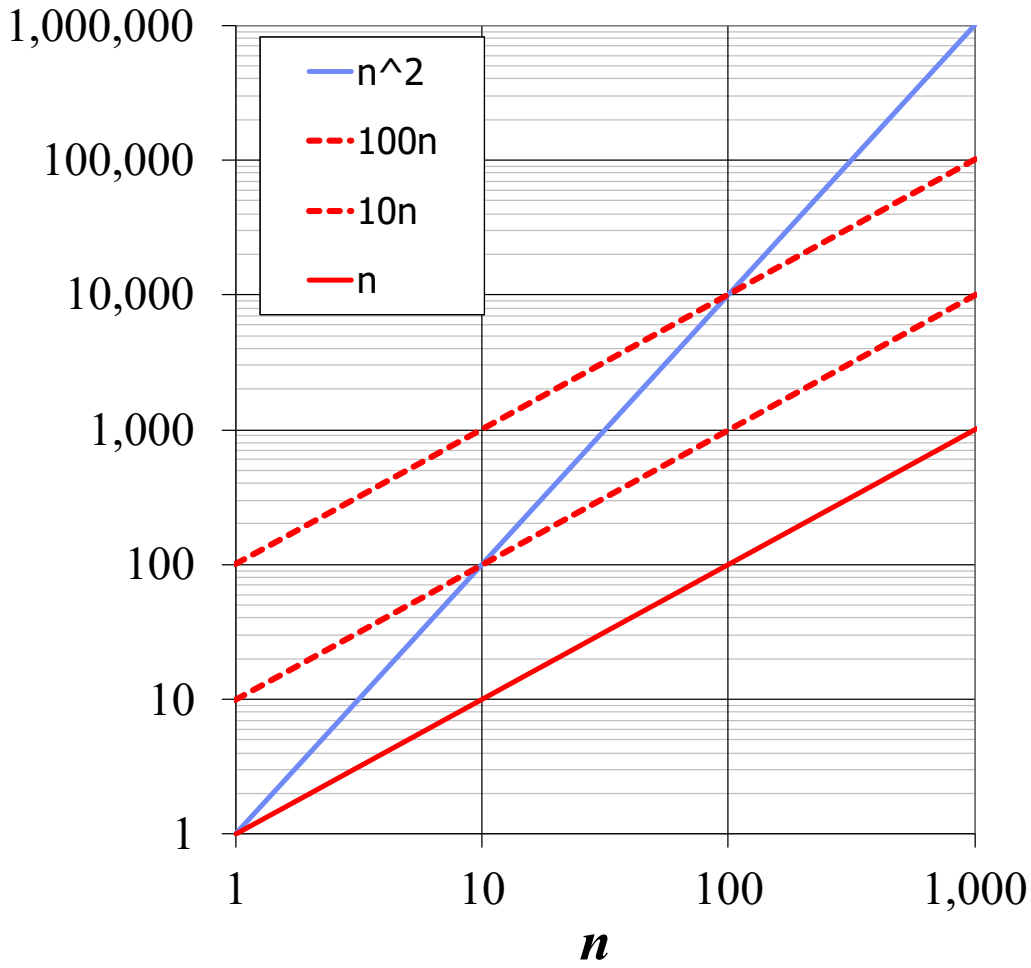
Consider Big-O of ***arrayMax***

- ***arrayMax*** executes at most $7n$ primitive operations
  - constant factor "7" can be disregarded from 7*n*
  - lower order terms can be disregarded
    - include them to try
  - Find *n* and *c* where
    - $n \leq c \cdot n$ for $n \geq n_0$
    - $c \geq 1$, $n_0 \geq 0$

- We say that algorithm ***arrayMax*** "runs in $O(n)$ time"

Big-O Example

- Example: the function n^2 is not $O(n)$
  - $n^2 \leq cn$
  - $n \geq n_0$
  - Definition cannot be satisfied since c is a constant



Big-O and Growth Rate

- Big-O notation gives an *upper bound* on the *growth rate* of a function
- "$f(n)$ is $O(g(n))$" means the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- Big-O notation ranks functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Big-O Rules

- If $f(n)$ is a polynomial of degree d, then $f(n)$ is $O(n^d)$
  - drop lower-order terms
  - drop constant factors (coefficients)
- Use the smallest possible class of functions
  - "$2n$ is $O(n)$" instead of "$2n$ is $O(n^2)$"
- Use the simplest expression of the class
  - "$3n + 5$ is $O(n)$" instead of "$3n + 5$ is $O(3n)$"

More Big-O Examples

$7n - 2$

$7n - 2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n - 2 \leq c \cdot n$ for $n \geq n_0$

true for $c = 7$ and $n_0 = 1$

$3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

true for $c = 4$ and $n_0 = 21$

$3 \log n + 5$

$3 \log n + 5$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \cdot \log n$ for $n \geq n_0$

true for $c = 8$ and $n_0 = 2$

More Big-O Examples

$7n - 2$

$7n - 2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n - 2 \leq c \cdot n$ for $n \geq n_0$

true for $c = 7$ and $n_0 = 1$

$3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 +$

true for $c = 4$ and $n_0 = 21$

$3 \log n + 5$

$3 \log n + 5$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5$

true for $c = 8$ and $n_0 = 2$

Plan for finding *g* in "*f*(*n*) is O(*g*(*n*))"

Drop lower-order terms from *f*

Drop constant factors from *f*

Tentatively:

- find "smallest" class of functions
- use "simplest" expression of class
- find a value for *c*: e.g. look at coefficients in *f* incl. lower-order terms—sum of positives is usually good start
- work out small n_0 from *c* algebraically or "by eye"